

FITBUDDY AI

AI-Powered Personalized Fitness Plan Generator Using Gemini 2.5

COMPLETE PROJECT DOCUMENTATION

Technology Stack	FastAPI • Jinja2 • SQLite • SQLAlchemy • Google Gemini 2.5 Flash
Interface	HTML • CSS • Jinja2 templates
Development Environment	Python 3.11 • Visual Studio Code • Modern Web Browser
Documentation Basis	Reference project documentation + supplied VS Code project archive + runtime screenshots

Prepared as a submission-ready project report with implementation details and one-by-one runtime output screenshots.

TABLE OF CONTENTS

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Scope of the Project
6. System Requirements
7. Technology Stack
8. System Architecture
9. Project Folder Structure
10. Module Description
11. User Inputs and Data Model
12. AI Prompt and Gemini Integration
13. Working Process / System Flow
14. API Endpoints
15. Database and Data Persistence
16. Security and Configuration
17. Error Handling and Fallback Behaviour
18. User Interface Design
19. Advantages
20. Limitations
21. Future Enhancements
22. Testing and Validation
23. Installation and Execution
24. Expected Output
25. Runtime Screenshots – One by One
26. Conclusion
27. Project Summary
28. Safety Disclaimer
- Appendix A. Supplied Source Files Reviewed

1. ABSTRACT

FitBuddy AI is a web application that uses Generative AI to create a personalized 7-day fitness plan from user-provided information. The supplied reference documentation describes a broader fitness-profile workflow, while the supplied implementation archive realizes a focused version using FastAPI, Jinja2 templates, SQLite/SQLAlchemy, and Google Gemini 2.5 Flash. The application collects name, user ID, age, weight, fitness goal, and workout intensity, generates a workout plan and a nutrition/recovery tip, stores the result, and allows the user to request a revised plan using feedback.

The implementation also includes an administrative view for stored users and their original/updated plans, plus FastAPI Swagger documentation. The screenshots included in this report document the actual runtime flow observed in the supplied project: home page → generated plan → plan continuation → nutrition/feedback → admin dashboard → API documentation.

2. INTRODUCTION

Maintaining a consistent fitness routine requires planning and personalization. The reference documentation identifies the difficulty of matching workout schedules to individual goals, time, fitness level, equipment, and dietary preferences. FitBuddy AI demonstrates how a Generative AI service can be connected to a browser-based application so that a user can submit profile information and receive dynamically generated fitness guidance.

The implemented project uses a client-server architecture. The browser submits form data to FastAPI, the backend prepares a prompt for Gemini, the generated text is returned to the application, and the result is rendered using Jinja2 templates. SQLite provides lightweight local persistence for user records and plan revisions.

3. PROBLEM STATEMENT

Predefined fitness routines may not reflect every user's requirements. Manually creating a weekly schedule can also take time and requires knowledge of exercise selection, workload, recovery, and nutrition. FitBuddy AI addresses this by accepting user-specific fitness information and using Generative AI to create a structured, readable plan automatically.

4. OBJECTIVES

- To develop an AI-assisted fitness planning web application.
- To collect user fitness information through a simple browser interface.
- To integrate Google Gemini Generative AI into a FastAPI application.
- To generate a practical 7-day workout plan.
- To provide a nutrition/recovery tip.
- To allow feedback-based plan revision.
- To store generated plans locally for later viewing in an admin dashboard.
- To provide API documentation and a health-check endpoint.
- To demonstrate a complete full-stack Generative AI workflow.

5. SCOPE OF THE PROJECT

Implemented scope: The supplied code currently focuses on six user inputs—name, user ID, age, weight, fitness goal, and workout intensity—and produces a 7-day workout plan plus a nutrition/recovery tip. The user can submit feedback to revise the plan. Records are stored in SQLite and displayed through an admin

dashboard.

Reference-document scope: The reference documentation additionally describes gender, height, fitness level, workout preference, days per week, workout duration, equipment, dietary preference, and additional requirements. Those fields are part of the reference design but are not present in the supplied implementation archive's form.

6. SYSTEM REQUIREMENTS

Hardware: The reference documentation specifies Intel Core i3/equivalent, 4 GB RAM or higher, 1 GB available storage, internet connection, keyboard/mouse, and monitor; 8 GB RAM or higher is recommended.

Software: Windows/Linux/macOS, Python 3.10+, Visual Studio Code, a modern web browser, internet connection, and a Google Gemini API key when real Gemini generation is enabled.

7. TECHNOLOGY STACK

Layer	Technology / Library	Role
Web UI	HTML + CSS + Jinja2	Forms, styling, result pages and admin view
Backend	Python + FastAPI	Routes, request handling and API documentation
Templates	Jinja2	Server-side HTML rendering
Database	SQLite + SQLAlchemy	Local user/plan persistence
AI	Google Gemini 2.5 Flash via google-genai	Plan, tip and feedback-revision generation
Configuration	python-dotenv + .env	API key and runtime settings
Server	Uvicorn	Local ASGI application server
API Docs	FastAPI Swagger UI	Interactive endpoint documentation

8. SYSTEM ARCHITECTURE

The implemented application follows a simple browser-to-backend-to-AI architecture. The main runtime sequence is:

1	User	Enters name, user ID, age, weight, goal and intensity.
2	FastAPI /generate-workout	Receives the form submission.
3	Gemini service logic	Builds a prompt and calls Gemini 2.5 Flash when configured.
4	Fallback layer	Uses the built-in demo plan/tip if demo mode is enabled, Gemini is not configured, or a generation call fails.
5	SQLite / SQLAlchemy	Stores the user and generated plan.
6	Jinja2 result page	Displays the personalized plan and nutrition/recovery tip.
7	Feedback route	Uses feedback to create a revised plan and stores it as updated_plan.
8	Admin / Swagger	Provides stored-record review and interactive API documentation.

Conceptual flow: Browser → FastAPI → Gemini 2.5 Flash → FastAPI → Jinja2 result page; SQLite runs alongside the backend for persistence.

9. PROJECT FOLDER STRUCTURE

```
FitBuddy_Gemini25/
├── app/
│   ├── main.py
│   └── templates/
│       ├── index.html
│       ├── result.html
│       ├── error.html
│       └── all_users.html
├── start_windows.bat
├── requirements.txt
├── .env.example
├── .gitignore
└── README.md
```

The supplied archive contains the application entry point, four Jinja2 templates, dependency configuration, environment example, Git ignore rules, Windows launcher, and README. The backend creates the local SQLite database file fitbuddy.db at runtime.

10. MODULE DESCRIPTION

10.1 Backend Module – app/main.py

- Loads environment variables with python-dotenv.
- Creates the FastAPI application and Jinja2 template environment.
- Defines the SQLAlchemy User model and creates the SQLite database tables.
- Initializes the Gemini client from GEMINI_API_KEY or GOOGLE_API_KEY.
- Implements plan generation, nutrition/recovery tip generation and feedback-based plan updating.
- Defines the browser routes and health endpoint.
- Returns HTML responses rendered by Jinja2.

10.2 Frontend / Template Module

- index.html provides the data-entry form and navigation to the admin dashboard and API docs.
- result.html shows the user summary, 7-day plan, nutrition/recovery tip and feedback form.
- all_users.html displays stored users and their original/updated plans.
- error.html displays a simple user-facing error when a feedback request references an unknown user ID.

10.3 AI Integration Module

AI communication is implemented inside main.py through the google-genai client. The model name is read from GEMINI_MODEL and defaults to gemini-2.5-flash. The project also contains a DEMO_MODE setting and built-in fallback generation so the UI can be demonstrated without a working Gemini response.

11. USER INPUTS AND DATA MODEL

Field	Type / Storage	Purpose
user_id	String, unique	Identifies the stored user record
username	String	User display name
age	Integer	Age used in the generation prompt
weight	String	Weight value displayed in the profile and prompt
goal	String	Fitness goal such as weight loss or muscle gain
intensity	String	Workout intensity such as low/medium/high

Field	Type / Storage	Purpose
original_plan	Text	First generated 7-day plan
updated_plan	Text, nullable	Revised plan after feedback
nutrition_tip	Text	Nutrition/recovery guidance

12. AI PROMPT AND GEMINI INTEGRATION

The implementation builds three kinds of prompts: a new-plan prompt, a nutrition/recovery-tip prompt, and a feedback-revision prompt. The new-plan prompt identifies FitBuddy as an AI fitness planning assistant and asks for a practical personalized 7-day plan using the user's name, age, weight, goal and intensity. It requests day-by-day focus, warm-up, exercises with sets/ reps or duration, and cooldown/recovery guidance.

The feedback prompt supplies the original plan and the user feedback and requests a complete revised 7-day plan. The code keeps the Gemini call behind a small helper function, making the AI integration easy to configure through environment variables.

Configuration used by the supplied project:

```
GEMINI_API_KEY=PASTE_YOUR_KEY_HERE
DEMO_MODE=false
GEMINI_MODEL=gemini-2.5-flash
```

13. WORKING PROCESS / SYSTEM FLOW

- Step 1 – Open the FitBuddy home page.
- Step 2 – Enter name, user ID, age, weight, fitness goal and workout intensity.
- Step 3 – Submit the Generate 7-Day Plan form.
- Step 4 – FastAPI receives the form at POST /generate-workout.
- Step 5 – The backend creates the AI prompt and calls Gemini when configured.
- Step 6 – If demo mode is active, no Gemini call is made; if Gemini is unavailable or errors, the built-in fallback is used.
- Step 7 – The plan and nutrition/recovery tip are stored in SQLite.
- Step 8 – result.html displays the generated plan.
- Step 9 – The user can submit feedback using POST /submit-feedback.
- Step 10 – The revised plan is stored as updated_plan and displayed.
- Step 11 – The admin dashboard shows stored original and updated plans.
- Step 12 – /docs exposes FastAPI's interactive Swagger documentation.

14. API ENDPOINTS

Method	Endpoint	Purpose
GET	/	Displays the FitBuddy home form
POST	/generate-workout	Generates and stores a 7-day plan
POST	/submit-feedback	Creates and stores a revised plan
GET	/view-all-users	Displays stored users and plans
GET	/health	Returns service/model configuration status
GET	/docs	FastAPI Swagger UI
GET	/openapi.json	FastAPI OpenAPI schema

Method	Endpoint	Purpose
GET	/docs-link	Redirects to /docs

The reference documentation names `/api/generate-plan` as its API route, but the supplied implementation uses browser form routes `/generate-workout` and `/submit-feedback`. The screenshots and code in this report document the implemented routes.

15. DATABASE AND DATA PERSISTENCE

SQLite is used as the local database. SQLAlchemy defines a User table with a unique `user_id` and fields for profile values, the original plan, the updated plan, and the nutrition tip. When a plan is generated for an existing user ID, the record is updated; otherwise a new record is inserted. This supports the admin dashboard shown in Figure 5.

16. SECURITY AND CONFIGURATION

- The Gemini API key is loaded from an environment file rather than embedded in HTML or client-side JavaScript.
- The supplied `.gitignore` excludes `.env`, virtual environments, Python cache files and `fitbuddy.db`.
- The `.env` file should never be committed to a public repository.
- The reference documentation also recommends environment-based API-key management.

Recommended .env example

```
GEMINI_API_KEY=your_real_key
DEMO_MODE=false
GEMINI_MODEL=gemini-2.5-flash
```

17. ERROR HANDLING AND FALLBACK BEHAVIOUR

The application handles missing Gemini configuration and generation exceptions by falling back to the built-in demo plan/tip. This keeps the application usable for interface demonstrations. The feedback route checks whether the user ID exists and renders `error.html` if no matching user record is found. The README also documents common setup problems such as missing dependencies, incorrect Uvicorn module paths, occupied ports, and Gemini authentication/model errors.

18. USER INTERFACE DESIGN

The UI uses a dark theme with teal accents. The home page uses a two-column form on wider screens and collapses to a single column on smaller screens. The result page separates the user summary, workout plan, nutrition/recovery tip and feedback form into cards. The admin page presents original and updated plans side by side, while FastAPI's Swagger UI provides a standard interactive API documentation experience.

19. ADVANTAGES

- Personalized AI-generated workout guidance.
- Simple browser-based workflow.
- Local persistence using SQLite.
- Feedback-based plan revision.
- Built-in demo fallback for UI testing.
- Interactive Swagger API documentation.
- Straightforward FastAPI/Jinja2 architecture suitable for learning and demonstration.

20. LIMITATIONS

- Generated fitness guidance depends on the information supplied by the user.
- The current implementation collects fewer profile fields than the broader reference specification.
- AI-generated recommendations are not a substitute for professional medical or fitness advice.
- Real Gemini generation requires valid API configuration and internet connectivity.
- SQLite is appropriate for a local project/demo but may not be the preferred database for a large multi-user deployment.

21. FUTURE ENHANCEMENTS

- User authentication and authorization.
- Richer fitness profile fields from the reference design.
- Workout history and progress tracking.
- BMI/calorie features where appropriate.
- Exercise demonstration media.
- More advanced AI-based workout modification.
- Voice interaction and mobile application.
- Reminders, charts and wearable integration.
- Production database and deployment.
- Multi-language support.

22. TESTING AND VALIDATION

Test	Expected result	Observed evidence
Home page	Form loads and fields are visible	Figure 1
Plan generation	7-day plan is displayed	Figures 2–3
Nutrition/recovery	Tip is shown	Figure 4
Feedback UI	Feedback box and update button are available	Figure 4
Persistence/admin	User and original/updated plans can be viewed	Figure 5
API documentation	Swagger UI lists endpoints	Figure 6
Health endpoint	Returns status information	Endpoint defined in main.py

The screenshots demonstrate successful local execution at <http://127.0.0.1:8000>. The displayed plan in the screenshots follows the built-in demo-plan structure present in the supplied code, while the same application can call Gemini 2.5 Flash when configured.

23. INSTALLATION AND EXECUTION

The supplied README provides the following Windows-oriented setup flow:

Step 1: Open the FitBuddy_Gemini25 folder in Visual Studio Code.

Step 2: Create a Python 3.11 virtual environment.

Step 3: Activate the environment.

Step 4: Install dependencies with `pip install -r requirements.txt`.

Step 5: Copy .env.example to .env and add the Gemini API key.

Step 6: Keep DEMO_MODE=false for real Gemini output, or use DEMO_MODE=true for UI/demo testing.

Step 7: Run python -m uvicorn app.main:app --reload.

Step 8: Open http://127.0.0.1:8000 in the browser.

Step 9: Use /docs for Swagger UI and /view-all-users for the admin dashboard.

Windows launcher

```
@echo off
py -3.11 -m venv .venv
call .venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements.txt
python -m uvicorn app.main:app --reload
```

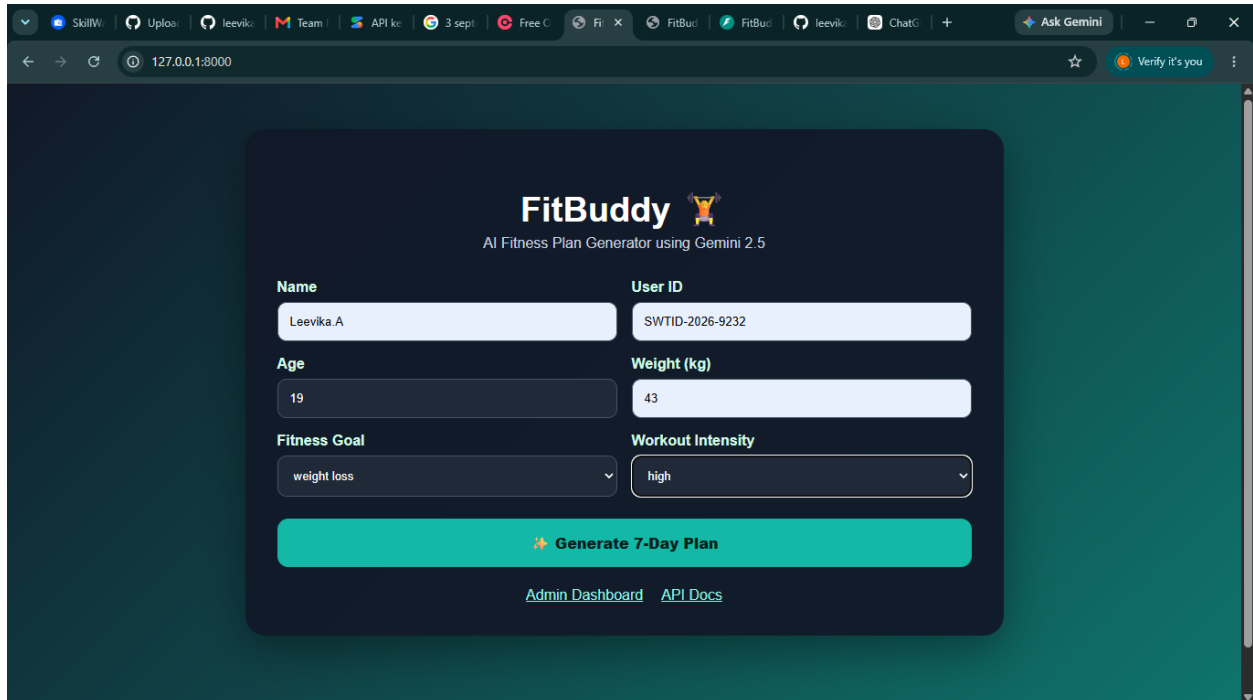
24. EXPECTED OUTPUT

A successful run presents a FitBuddy home page, a generated 7-day workout plan, a nutrition/recovery tip, a feedback-based update area, an admin dashboard with persisted records, and interactive FastAPI documentation. The actual runtime screenshots supplied with this report show these stages in sequence.

25. RUNTIME SCREENSHOTS – ONE BY ONE

The following screenshots are attached in execution order so that the report clearly shows what appears first, what follows, and where each page belongs in the application workflow.

1. Figure 1. FitBuddy home page and user input form.



The screenshot shows a web browser window displaying the FitBuddy home page. The browser's address bar shows the URL 127.0.0.1:8000. The page features a dark blue background with a central white card containing the FitBuddy logo and the text "AI Fitness Plan Generator using Gemini 2.5". Below this, there is a user input form with the following fields:

Name	User ID
Leevika A	SWTID-2026-9232
Age	Weight (kg)
19	43
Fitness Goal	Workout Intensity
weight loss	high

Below the form is a large blue button labeled "Generate 7-Day Plan". At the bottom of the card, there are two links: "Admin Dashboard" and "API Docs".

Figure 1. FitBuddy home page and user input form.

2. Figure 2. Generated personalized plan – user summary and beginning of the 7-day plan.

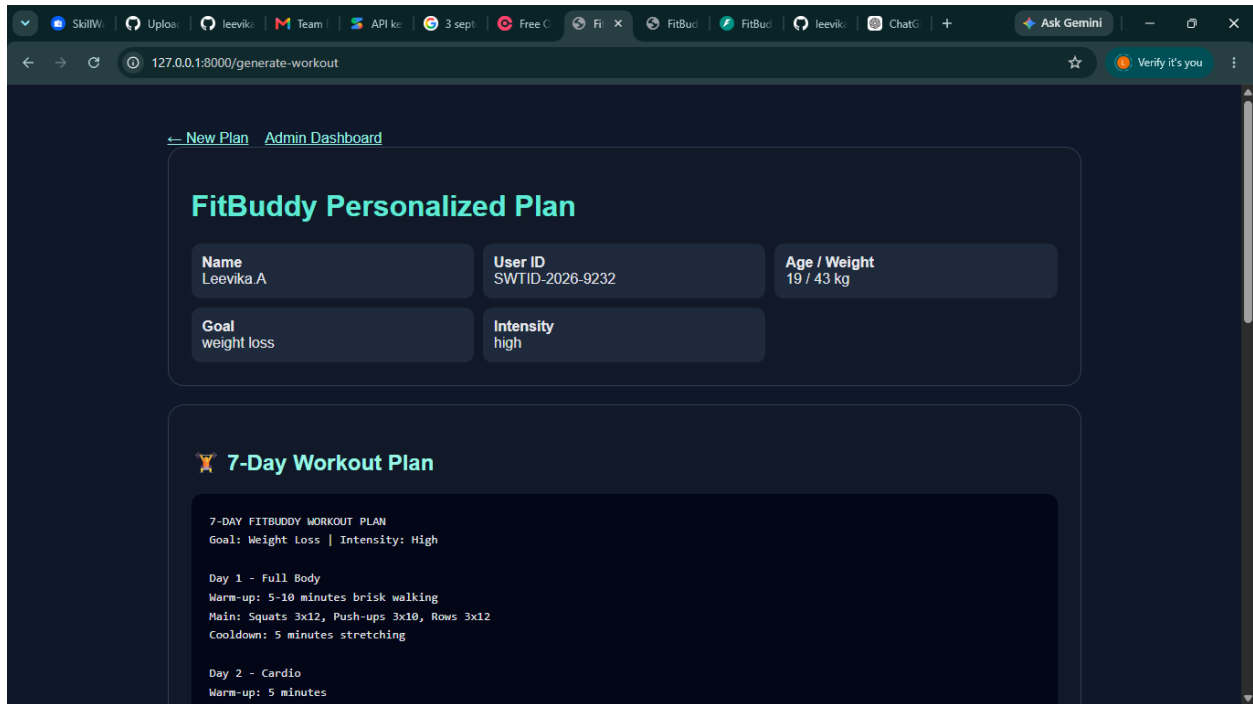


Figure 2. Generated personalized plan – user summary and beginning of the 7-day plan.

3. Figure 3. Generated personalized plan – continuation of the 7-day workout schedule.

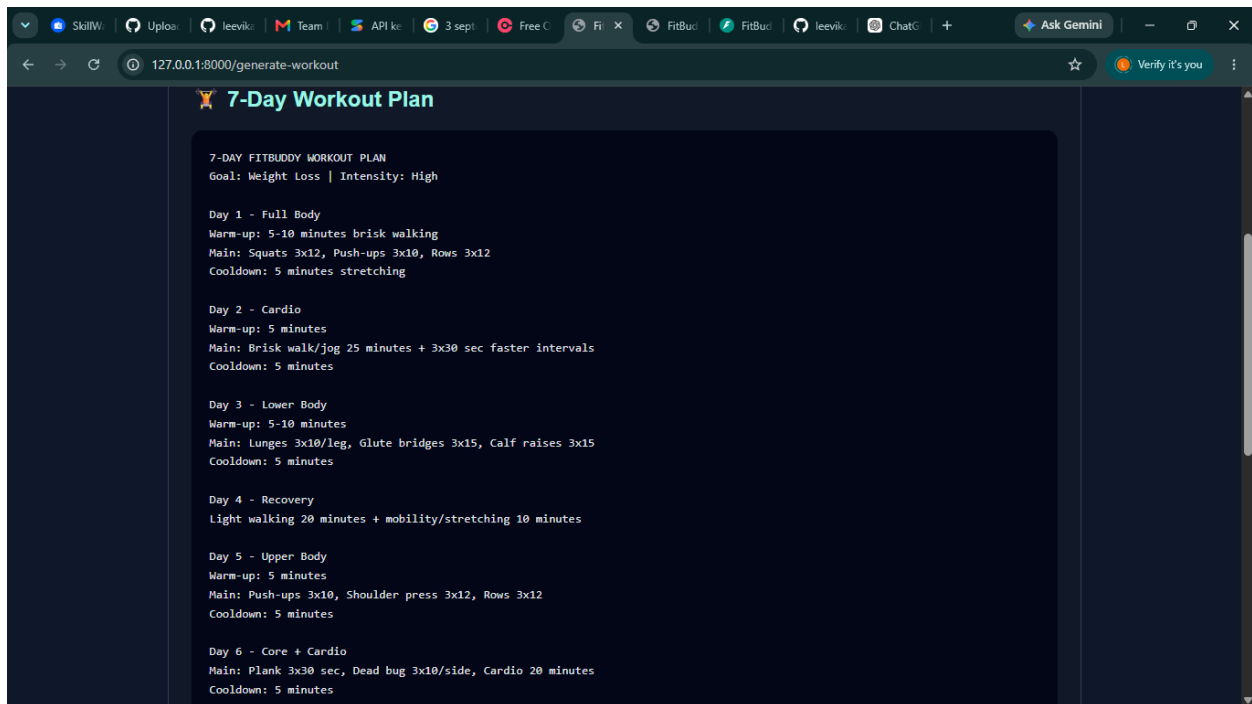


Figure 3. Generated personalized plan – continuation of the 7-day workout schedule.

4. Figure 4. Nutrition/recovery tip and feedback-based plan update section.

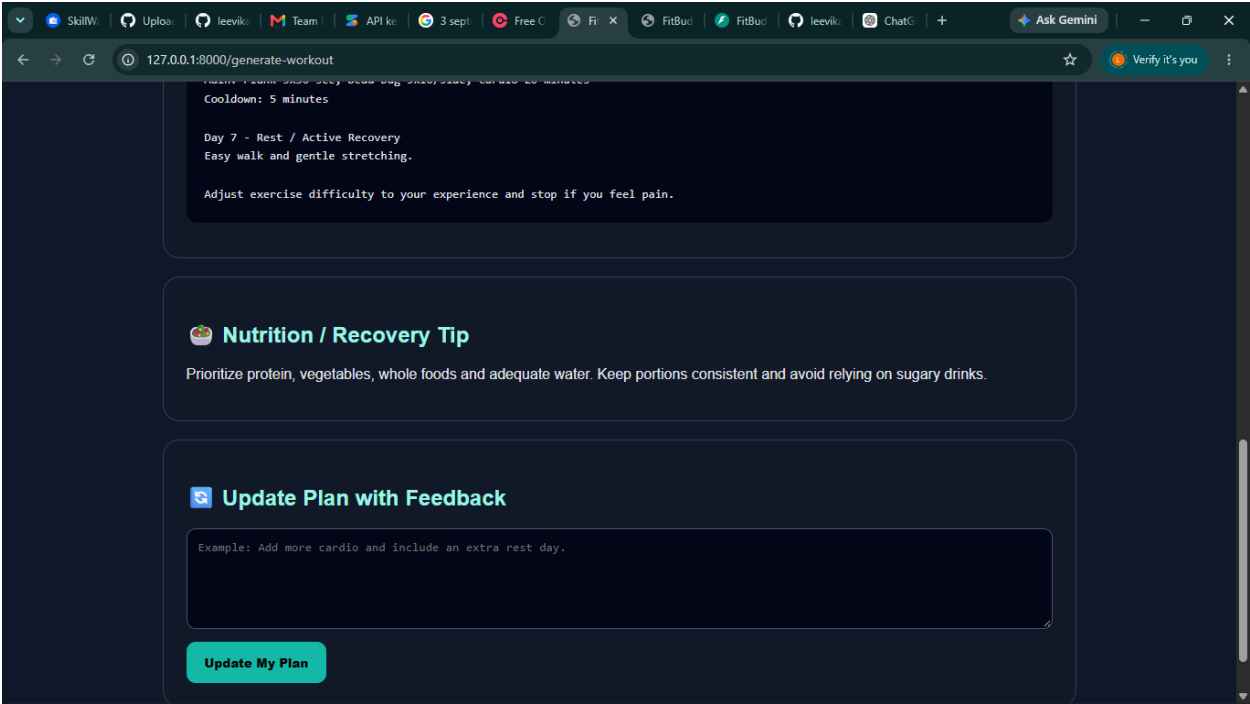


Figure 4. Nutrition/recovery tip and feedback-based plan update section.

5. Figure 5. Admin dashboard showing stored user data and original/updated plans.

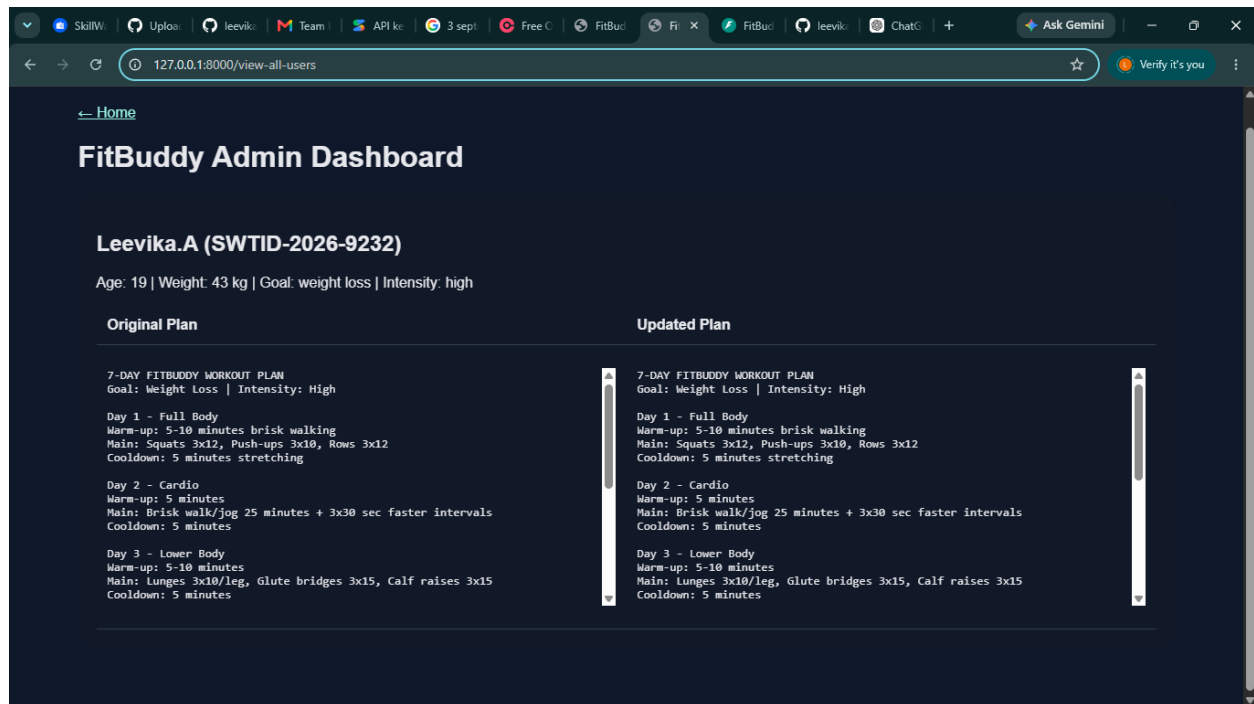


Figure 5. Admin dashboard showing stored user data and original/updated plans.

6. Figure 6. FastAPI Swagger UI / API documentation page.

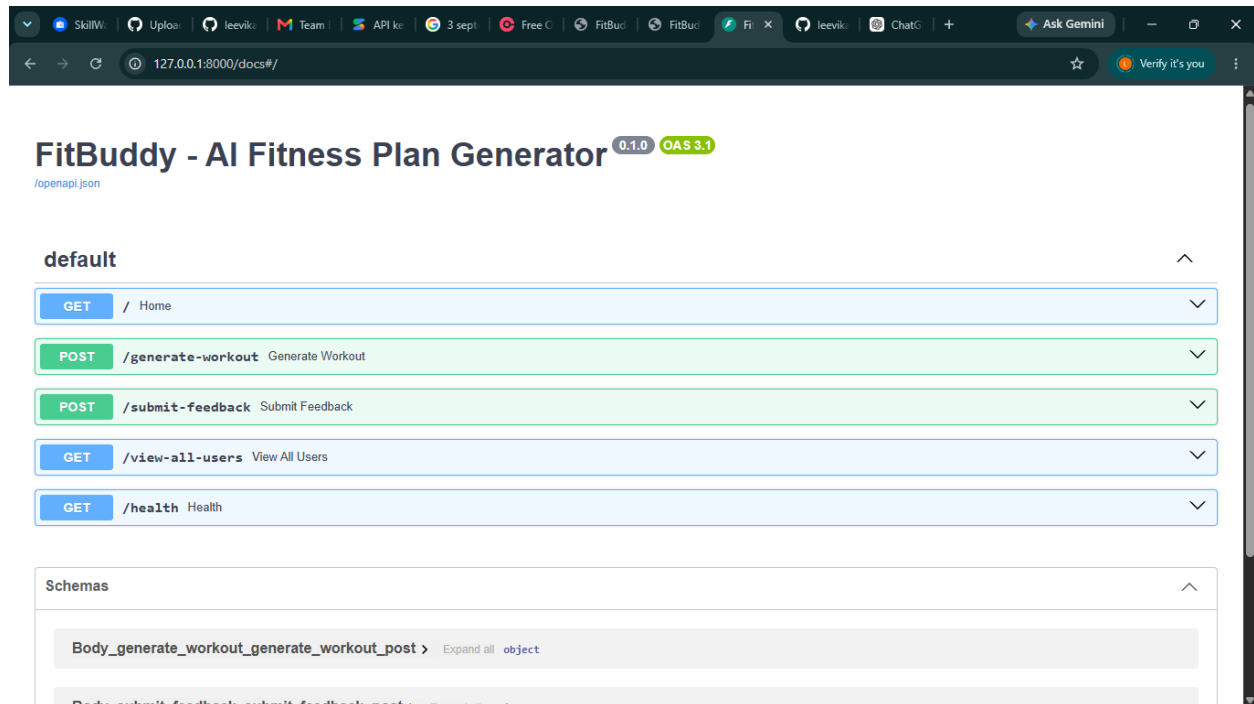


Figure 6. FastAPI Swagger UI / API documentation page.

26. CONCLUSION

FitBuddy AI demonstrates a complete Generative AI application workflow using a web interface, FastAPI backend, SQLite persistence, and Google Gemini 2.5 Flash integration. The supplied implementation adds practical project features beyond simple generation: feedback-based plan revision, stored user records, an admin dashboard, health status reporting, and interactive API documentation. The runtime screenshots confirm that the local application reaches the main user-facing, administrative, and API-documentation pages.

27. PROJECT SUMMARY

Category	Implemented project
Project Name	FitBuddy – AI Fitness Plan Generator
Domain	Fitness / Generative AI
Frontend	HTML, CSS, Jinja2 templates
Backend	Python FastAPI
AI	Google Gemini 2.5 Flash via google-genai
Database	SQLite with SQLAlchemy
Architecture	Client–server / server-rendered web application
Main output	Personalized 7-day workout plan + nutrition/recovery tip
Feedback	Revised plan generation and storage
Admin	Stored user and plan comparison
API docs	FastAPI Swagger UI
IDE	Visual Studio Code

28. SAFETY DISCLAIMER

FitBuddy AI provides general informational fitness guidance generated using artificial intelligence. The generated content is not a substitute for medical diagnosis, treatment, or individualized professional advice. Users with injuries, medical conditions, pregnancy, or other health concerns should consult an appropriate qualified healthcare professional before beginning or changing a fitness program. The reference documentation explicitly includes this safety framing, and the supplied AI prompt also instructs the model not to provide medical diagnosis or treatment.

APPENDIX A. SUPPLIED SOURCE FILES REVIEWED

- FitBuddy_Gemini25/app/main.py
- FitBuddy_Gemini25/templates/index.html
- FitBuddy_Gemini25/templates/result.html
- FitBuddy_Gemini25/templates/error.html
- FitBuddy_Gemini25/templates/all_users.html
- FitBuddy_Gemini25/requirements.txt
- FitBuddy_Gemini25/.env.example
- FitBuddy_Gemini25/.gitignore
- FitBuddy_Gemini25/README.md
- FitBuddy_Gemini25/start_windows.bat

This report was prepared from the supplied reference documentation, the supplied FitBuddy project archive, and the six supplied runtime screenshots. Where the reference documentation and implementation differ, this report identifies the implemented behaviour rather than silently replacing it with the reference design.